

**Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Херсонский государственный педагогический университет»**

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Современные операционные системы и базы данных»
на тему:
«PowerShell Сценарий проверки сайта»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверила
Преподаватель:
доцент, к.н., Галлини Надежда
Игоревна

Оценка: _____
Дата: 19.09.2026

Херсон, 2026

Цель работы

Сценарий сообщает статус либо причину ошибки, а студент объясняет отличие CMD, PowerShell и Windows Terminal.

CMD – программа на винде (исполняемый файл .exe), командная строка.

PS – PowerShell – интерпретатор командной строки в Виндоуз, улучшенная версия cmd.exe

Windows Terminal – приложение-контейнер, видимо какая-то «современная» от Windows 11.

Что потребуется

PowerShell и готовый учебный адрес. Выполнять в своей папке.

Или в директории на Linux.

Сценарий не изменяет системные службы.

Файлы этой работы находятся в папке «Примеры/02» внутри каталога «04 Практикум».

Скопируйте всю папку в собственный рабочий каталог C:\web-lab\02, сохранив вложенные папки. Исходные файлы курса оставьте в качестве образца.

Для работы 4 используется каталог Linux, указанный в инструкции.

Интересно, как соотносится тема работы «PowerShell – сценарий проверки сайта» с «папками для PS» и «каталогами в Linux». И непонятно какая связь с сайтом и каким сайтом.

Перед началом уточните, какая среда подготовлена: локальный компьютер, WSL или виртуальная машина. Для серверных СУБД преподаватель выдаёт адрес, порт, имя учебной базы и учётную запись. Значения в угловых скобках в командах являются заменяемыми параметрами, их нельзя вводить буквально.

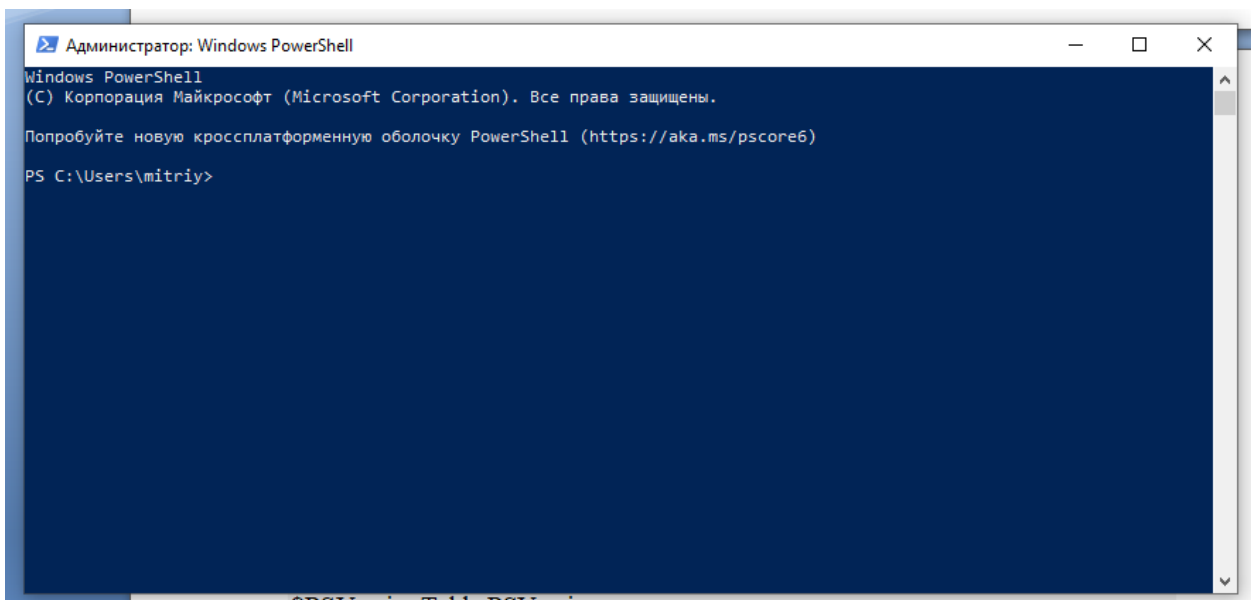
В работе будет использоваться Windows 10, PS, Git Bash.

Порядок выполнения

Шаг 1. Оболочка и рабочая папка

Откройте PowerShell.

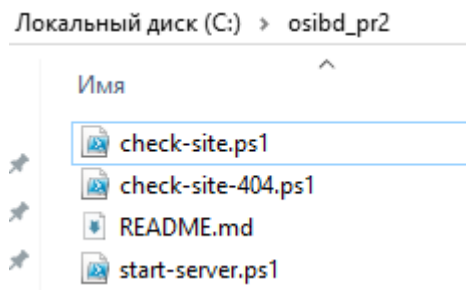
Открыл PS.



Скопируйте файлы из **Примеры/02** в C:\web-lab\02. Команды этой работы вводятся в PowerShell, а не в CMD.

Есть стойкое желание вводить команды в Git Bash (линуксоподобный), а не в майкрософтовский PS.

Файлы скопированы в директорию /c/osibd_pr2/.



```
Set-Location C:\web-lab\02
```

```
$PSVersionTable.PSVersion
```

```
Get-ChildItem
```

Set-Location – это, наверное, как `cd` в Git Bash. Подтверждено.

Посмотрел версию моего PS.

Get-ChildItem, это аналог `ls` в Git Bash.

```
PS C:\Users\mitriy> Set-Location C:\osibd_pr2\
PS C:\osibd_pr2> $PSVersionTable.PSVersion

Major Minor Build Revision
-----
5      1      19041  5607

PS C:\osibd_pr2> Get-ChildItem

Каталог: C:\osibd_pr2

Mode                LastWriteTime         Length Name
----                -
-a----             18.09.2026    20:38         1632 check-site-404.ps1
-a----             18.09.2026    20:38          465 check-site.ps1
-a----             18.09.2026    20:38        3145 README.md
-a----             18.09.2026    20:38        2316 start-server.ps1

PS C:\osibd_pr2>
```

Проверка результата. Выведена версия PowerShell; в каталоге присутствует check-site.ps1.

Присутствует check-site.ps1 в каталоге/папке/директории.

Шаг 2. Объекты процессов и служб

Выведите небольшой список процессов. Name и Id являются свойствами объекта. В следующей команде оставьте имя учебного процесса, выданное преподавателем.

```
Get-Process | Select-Object -First 8 Name,Id
Get-Service | Select-Object -First 8 Name,Status
```

Проверка результата. Процессы имеют PID, службы имеют имя и состояние. Служба может работать без открытого окна терминала.

«Служба может работать без открытого окна терминала» звучит как очевидное замечание. Однако студент часто думает: «я запустил что-то в терминале — оно работает, закрыл терминал — всё пропало».

Вывел 4 процесса с айдишниками.

```
PS C:\osibd_pr2> Get-Process | Select-Object -First 4 Name,Id

Name                Id
----                --
AdGuardVpnSvc       2684
ApplicationFrameHost 3716
audiodg              3748
bash                 6180
```

Вывел 4 процесса с айдишниками и статусами.

```
PS C:\osibd_pr2> Get-Process | Select-Object -First 4 Name,Id,Status
```

Name	Id	Status
AdGuardVpnSvc	2684	
ApplicationFrameHost	3716	
audiodg	3748	
bash	6180	

Шаг 3. Проверка TCP

Уточните **адрес** и **порт** подготовленного учебного **HTTP-сервера**. Если сервер на другом компьютере, замените 127.0.0.1 его адресом.

Test-NetConnection 127.0.0.1 -Port 8080

```
PS C:\osibd_pr2> Test-NetConnection 127.0.0.1 -Port 8080
ПРЕДУПРЕЖДЕНИЕ: TCP connect to (127.0.0.1 : 8080) failed
```

ComputerName	: 127.0.0.1
RemoteAddress	: 127.0.0.1
RemotePort	: 8080
InterfaceAlias	: Loopback Pseudo-Interface 1
SourceAddress	: 127.0.0.1
PingSucceeded	: True
PingReplyDetails (RTT)	: 0 ms
TcpTestSucceeded	: False

Проверка результата. **TcpTestSucceeded=True** означает доступность TCP-порта. При False сначала проверьте адрес и **запуск сервера**.

Администратор: Windows PowerShell

```
учебный сервер запущен на http://127.0.0.1:8080/
Нажмите Ctrl+C для остановки.
```

Сервер запущен.

```
PS C:\osibd_pr2> Test-NetConnection 127.0.0.1 -Port 8080
```

ComputerName	: 127.0.0.1
RemoteAddress	: 127.0.0.1
RemotePort	: 8080
InterfaceAlias	: Loopback Pseudo-Interface 1
SourceAddress	: 127.0.0.1
TcpTestSucceeded	: True

TCP-порт доступен, потому что вывело True (истина).

Шаг 4. Чтение сценария

Откройте check-site.ps1 в текстовом редакторе.

Найдите `param`, `Invoke-WebRequest`, `try` и `catch`.

Параметр `Url` задаёт адрес, `try` содержит запрос, `catch` формирует сведения об ошибке.

```
check-site.ps1 X
check-site.ps1
1 # check-site.ps1
2 # Сценарий проверки сайта: сообщает статус либо причину ошибки
3 param([string]$Url = "http://127.0.0.1:8080/") # param
4
5 $ErrorActionPreference = 'Stop'
6
7 # try ↓
8 try {
9     # Invoke-WebRequest ниже ↓
10    $response = Invoke-WebRequest -Uri $Url -UseBasicParsing -TimeoutSec 5
11    [pscustomobject]@{Url=$Url; Status=[int]$response.StatusCode; Time=Get-Date}
12 # catch ↓
13 } catch {
14     [pscustomobject]@{Url=$Url; Error=$_.Exception.Message; Time=Get-Date}
15 }
16
17 # Параметр Url -> адрес, try содержит запрос, catch формирует сведения об ошибке
18
```

Get-Content .\check-site.ps1

Get-Content – это аналог `cat` в Git Bash.

Проверка результата.

В сценарии нет остановки служб.

Поле Status заполняется для успешного запроса, поле Error сообщает о сбое.

Администратор: Windows PowerShell

```
PS C:\osibd_pr2> Get-Content .\check-site.ps1
# check-site.ps1
# Сценарий проверки сайта: сообщает статус либо причину ошибки
param([string]$Url = "http://127.0.0.1:8080/") # param

$ErrorActionPreference = 'Stop'

# try ↓
try {
    # Invoke-WebRequest ниже ↓
    $response = Invoke-WebRequest -Uri $Url -UseBasicParsing -TimeoutSec 5
    [pscustomobject]@{Url=$Url; Status=[int]$response.StatusCode; Time=Get-Date}
# catch ↓
} catch {
    [pscustomobject]@{Url=$Url; Error=$_.Exception.Message; Time=Get-Date}
}

# Параметр Url -> адрес, try содержит запрос, catch формирует сведения об ошибке
PS C:\osibd_pr2>
```

Шаг 5. Запуск с правильным адресом

Выполните сценарий оператором `&`. Если политика среды запрещает запуск `.ps1`, выполните строки примера интерактивно по порядку с заданным `$Url`; системную политику не меняйте.

```
&.\check-site.ps1 -Url http://127.0.0.1:8080/
```

Проверка результата. При исправном сервере вывод содержит `Url`, `Status` и `Time`. Зафиксируйте адрес вместе со статусом.

```
check-site.ps1 X
check-site.ps1
1 # check-site.ps1
2 # Сценарий проверки сайта: сообщает статус либо причину ошибки
3 param([string]$Url = "http://127.0.0.1:8080/") # param
4
5 $ErrorActionPreference = 'Stop'
6
7 # try ↓
8 try {
9     $req = [System.Net.HttpWebRequest]::Create($Url)
10    $req.Proxy = $null
11    $req.Timeout = 5000
12    $resp = $req.GetResponse()
13
14    # Invoke-WebRequest ниже ↓ был, пришлось убрать, у меня PS 5.1, так как на W11 не иду
15    # $response = Invoke-WebRequest -Uri $Url -UseBasicParsing -TimeoutSec 5
16    [pscustomobject]@{Url=$Url; Status=[int]$response.StatusCode; Time=Get-Date}
17 # catch ↓
18 } catch {
19     [pscustomobject]@{Url=$Url; Error=$_.Exception.Message; Time=Get-Date}
20 }
21
22 # Параметр Url -> адрес, try содержит запрос, catch формирует сведения об ошибке
```

```
Администратор: Windows PowerShell
PS C:\osibd_pr2> & .\check-site.ps1 -Url http://127.0.0.1:8080/

Url                               Status Time
---                               -
http://127.0.0.1:8080/           0 19.09.2026 0:20:08
```

Адрес со статусом (0 – ошибок нет) зафиксирован.

Шаг 6. Сравнение ошибок

Повторите запуск с согласованным свободным портом. Затем верните правильный адрес и убедитесь, что проверка снова успешна.

```
&.\check-site.ps1 -Url http://127.0.0.1:8099/
```

Проверка результата. Появляется поле Error с причиной сбоя. Опишите разницу между доступностью TCP и успешным HTTP-ответом.

Проверка на порту :8100

```
Администратор: Windows PowerShell
PS C:\osibd_pr2> & .\check-site.ps1 -Uri http://127.0.0.1:8100/

Url          Error          Time
----          -
http://127.0.0.1:8100/ Исключение при вызове "GetResponse" с "0" аргументами: "Невозможно соединиться с удаленным сервером" 19.09.2026 0:23:41

PS C:\osibd_pr2> & .\check-site.ps1 -Uri http://127.0.0.1:8080/

Url          Status Time
----          -
http://127.0.0.1:8080/      0 19.09.2026 0:24:39

PS C:\osibd_pr2>
```

На :8100 сервера нет, на :8080 есть.

Исходные файлы для сверки

check-site.ps1

```
param([string]$Url = "http://127.0.0.1:8080/")
$ErrorActionPreference = 'Stop'
try {
    $response = Invoke-WebRequest -Uri $Url -UseBasicParsing -TimeoutSec 5
    [pscustomobject]@{Url=$Url; Status=[int]$response.StatusCode; Time=Get-Date}
} catch {
    [pscustomobject]@{Url=$Url; Error=$_.Exception.Message; Time=Get-Date}
}
```

Идентично, лишь `Invoke-WebRequest` убран.

Если результат отличается

В некоторых версиях curl является псевдонимом. Для отдельной программы использовать curl.exe, для командлета – `Invoke-WebRequest`.

Читайте первую содержательную строку ошибки и проверьте текущую папку, имя файла, выбранное окружение и фактический адрес. После исправления повторите тот же шаг. Отдельно фиксируйте отсутствие соединения, HTTP-ошибку и пустой корректный результат: они требуют разных проверок.

Самостоятельное задание

Запустите сценарий для главной страницы и отсутствующей страницы одного сервера. Объясните, какое поле содержит сведения о неуспешном ответе.


```
PS C:\osibd_pr2> & .\check-site.ps1 -Url http://127.0.0.1:8080/guest
Url      Error
-----
http://127.0.0.1:8080/guest Исключение при вызове "GetResponse" с "0" аргументами: "Удаленный сервер возвратил ошибку: (404) Не найден." 19.09.2026 0
```

Несуществующая страница – ошибка 404 (Not Found).

```
PS C:\osibd_pr2> & .\check-site.ps1 -Url http://127.0.0.1:8080/
Url      Status Time
-----
http://127.0.0.1:8080/ 200 19.09.2026 0:30:44
```

Корневая страница существует.

```
PS C:\osibd_pr2> & .\check-site-404.ps1 -Url http://127.0.0.1:8080/
=== Проверка главной страницы ===

=== Проверка несуществующей страницы ===
Url      Status Time
-----
http://127.0.0.1:8080 200 19.09.2026 0:33:10
http://127.0.0.1:8080/nonexistent-page 404 19.09.2026 0:33:10
```

Успешный запуск дополнительного скрипта (сценария) для проверки статусов (цветные текстовые значения в терминале PS).

Требования к отчёту

1. ФИО, группа, номер работы и название технологии.
2. Версии среды и схема запуска: где выполняется сервер, по какому адресу открыт результат.
3. Нумерованные шаги с фактическими командами и коротким объяснением их назначения.
4. Не менее трёх подписанных скриншотов: подготовка или настройка, успешный результат, проверка изменения или ошибки. На снимках должны читаться важные поля и результат.
5. Результат самостоятельного задания и объяснение одной встретившейся ошибки.
6. Вывод: какую задачу решает технология и как проверить работоспособность приложения.

Приложите изменённые исходники и конфигурацию.

Не включайте .env, node_modules, пароли и токены.

Для скриншотов строки подключения скрывают секреты; сохраняйте видимыми имя учебной БД и несекретные параметры, необходимые для объяснения.

Для выполнения данной практической работы БД не потребовалась. В данной работе была проверка сайта, сервера, без использования CRUD и базы данных. По-сути мы лишь получали ответы на запросы к учебному серверу.

Защита скриптов

После выполнения работы обучающийся индивидуально защищает решение перед преподавателем. Для работ с HTML, SQL и настройками сервера защищаются соответствующие команды, запросы, разметка и конфигурация.

1. Демонстрация. Открыть собственные исходные файлы, запустить решение и показать результат основного и самостоятельного задания.

2. Объяснение. Пояснить скрипт `check-site.ps1`, параметры запроса и обработку исключений. Указать назначение ключевых строк, входные данные и получаемый результат.

3. Изменение при преподавателе. По его указанию изменить адрес проверяемого сайта или время ожидания и показать результат. Перед запуском объяснить ожидаемый результат.

4. Проверка ошибки. Показать один предусмотренный работой ошибочный случай, объяснить его причину и способ исправления.

Вопросы для защиты:

Как передаётся адрес сайта?

Как флаг `Y` к команде `X` через терминал.

Зачем нужен `try/catch`?

«Попробуй-поймай» - это такая конструкция в программировании для отлова ошибок, чтобы заменять стандартное поведение. Например, когда программа может нежелательно завершиться от «грубой» ошибки, а если в коде прописана «трай-кэтч», то программа, например, выводит особое сообщение и выполнение программы/скрипта продолжается.

Как отличить успешный ответ от ошибки подключения?

У успешного ответа код 200 (статус), у ошибок, например, 500 (сервер не отвечает), 404 – такой страницы нет. Здесь при проблеме с `Invoke` из-за низкой версии PS выводилось особое сообщение с текстом ошибки. Проблема была решена отказом от использования `Invoke`, доступного в более свежих версиях PowerShell. Кроме того, установлено, что PS отказывается завершаться по unix-привычной схеме через сочетание клавиш `CTRL+D`, `CTRL+C`. Что не удобно и не привычно.

В отчёт добавить подраздел «Защита скрипта»: внесённое изменение, результат повторного запуска и краткие ответы на вопросы преподавателя.